

Лабораторная работа № 4. Эмуляция и измерение задержек в глобальных сетях

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	6
2	Задание	9
3	Выполнение лабораторной работы	10
3.1	Запуск лабораторной топологии	10
3.2	Интерактивные эксперименты	12
3.2.1	Добавление/изменение задержки в эмулируемой глобальной сети	12
3.2.2	Изменение задержки в эмулируемой глобальной сети	15
3.2.3	Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети	15
3.2.4	Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети	16
3.2.5	Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети	18
3.2.6	Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети	19
3.3	Воспроизведение экспериментов	20
3.3.1	Предварительная подготовка	20
3.3.2	Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети	21
3.3.3	Задание для самостоятельной работы	26
4	Выводы	29

Список иллюстраций

1.1 Синтаксис	7
3.1 Подключение к виртуальной машине	10
3.2 Задание простейшей топологии	10
3.3 Информация на хостах h1 и h2	11
3.4 Проверка подключения между хостами	11
3.5 Добавление задержки в 100 мс	12
3.6 Проверка задержки в 100 мс	13
3.7 Добавление задержки в 100 мс на втором хосте	14
3.8 Проверка, что соединение имеет RTT в 200мс	14
3.9 Изменение задержки до 50 мс	15
3.10 Проверка, что задержка 100 мс	15
3.11 Восстановление конфигураций	16
3.12 Проверка, что соединение не имеет явной задержки	16
3.13 Проверка, что имеется задержка 100 мс со случайным отклонением ±10 мс	17
3.14 Восстановление конфигурации на h1	17
3.15 Проверка, что есть задержка в 100 мс с вариацией ±10 мс и значе- нием корреляции в 25%	18
3.16 Проверка, что есть задержка, которая распределена в диапазоне 100 мс ±20 мс	19
3.17 Завершение работы mininet	20
3.18 Обновление ПО	20
3.19 Установка пакета geeqie	21
3.20 Создание каталога expname	21
3.21 Создание каталога simple-delay	22
3.22 Создание скрипта lab_netem_i.py	22
3.23 Создание скрипта lab_netem_i.py	23
3.24 Задание прав доступа	23
3.25 Создание Makefile	23
3.26 График ping.png	24
3.27 Запуск	24
3.28 Удаление первой строчки	25
3.29 График ping.png с учетом изменений	25
3.30 Скрипт для вычисления на основе ping.dat	26
3.31 Запуск скрипта для вычисления	26
3.32 Внесение изменений в lab_netem_i.py	27

3.33 Запуск обновленного скрипта с выводом графика	27
3.34 Запуск script.py на основе обновленного ping.dat	28

Список таблиц

1 Цель работы

Основной целью работы является знакомство с NETEM — инструментом для тестирования производительности приложений в виртуальной сети, а также получение навыков проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.

Модуль NETEM входит в ядро ОС Linux и управляется при помощи команды `tc` из пакета `iproute2`. Команда `tc` выполняет все необходимые действия при работе с системой управления трафиком. Основной синтаксис `tc`, используемый с NETEM (рис. 1.1):

```

mininet@mininet-vm:~$ sudo tc qdisc [add|del|replace|change|show] dev dev_id root
t netem opts
Command "[add" is unknown, try "tc qdisc help".

Command 'replace' not found, but can be installed with:

sudo apt install mariadb-server-10.3

Command 'del' not found, did you mean:

command 'qdel' from deb gridengine-client (8.1.9+dfsg-9build2)
command 'qdel' from deb slurm-wlm-torque (19.05.5-1)
command 'el' from deb oneliner-el (0.3.6-8)
command 'wdel' from deb wput (0.6.2+git20130413-8)
command 'dep' from deb go-dep (0.5.4-3ubuntu0.1)
command 'hdel' from deb hfsutils (3.2.6-14)
command 'dex' from deb dex (0.8.0-2)
command 'dll' from deb brickos (0.9.0.dfsg-12.2)
command 'tel' from deb orville-write (2.55-3build1)
command 'mdel' from deb mtools (4.0.24-1)
command 'delv' from deb bind9-dnsutils (1:9.18.30-0ubuntu0.20.04.2)
command 'delp' from deb fp-utils-3.0.4 (3.0.4+dfsg-23)
command 'deal' from deb deal (3.1.9-12)

Try: sudo apt install <deb name>

Command 'show]' not found, did you mean:

command 'showq' from deb showq (0.4.1+git20180114-1)
command 'show' from deb mailutils-mh (1:3.7-2.1)
command 'show' from deb mmh (0.4-2)
command 'show' from deb nmh (1.7.1-6)

Try: sudo apt install <deb name>

Command 'change' not found, did you mean:

command 'charge' from deb emboss (6.6.0+dfsg-7ubuntu2)
command 'chage' from deb passwd (1:4.8.1-1ubuntu5.20.04.5)

Try: sudo apt install <deb name>

```

Рис. 1.1: Синтаксис

Здесь:

- `sudo` — включить выполнение команды с более высокими привилегиями безопасности;
- `tc` — команда, используемая для взаимодействия с NETEM;
- `qdisc` — дисциплина очереди (`qdisc`), представляющая собой набор правил, определяющих порядок, в котором обслуживаются пакеты, поступающие с выхода протокола IP.
- `[add | del | replace | change | show]` ([добавить | удалить | заменить | изменить | показать]): указание на использование одной из операций над `qdisc` (например, для добавления задержки на определенном интерфейсе исполь-

зуется опция `add`, а для изменения или удаления задержки на определенном интерфейсе используется опция `change` или `del` соответственно);

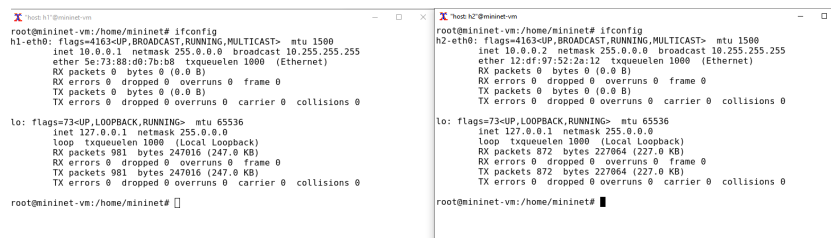
- `dev_id` — параметр указывает интерфейс, подлежащий эмуляции (в данном случае указан корневой интерфейс — `root`);
- `opts` — параметр указывает величину задержки, потери пакетов, дублирования, повреждения и др.

2 Задание

1. Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8.
2. Проведите интерактивные эксперименты по добавлению/изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети.
3. Реализуйте воспроизводимый эксперимент по заданию значения задержки в эмулируемой глобальной сети. Постройте график.
4. Самостоятельно реализуйте воспроизводимые эксперименты по изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети. Постройте графики

После введения этой команды запустятся терминалы двух хостов, коммутатора и контроллера. Терминалы коммутатора и контроллера можно закрыть.

3. На хостах h1 и h2 введем команду `ifconfig`, чтобы отобразить информацию, относящуюся к их сетевым интерфейсам и назначенным им IP-адресам. В дальнейшем при работе с NETEM и командой `tc` будут использоваться интерфейсы `h1-eth0` и `h2-eth0` (рис. 3.3):



```
root@mininet-vms:/home/mininet# ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 5e:73:88:d0:7b:b8 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 981 bytes 247016 (247.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 981 bytes 247016 (247.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vms:/home/mininet#

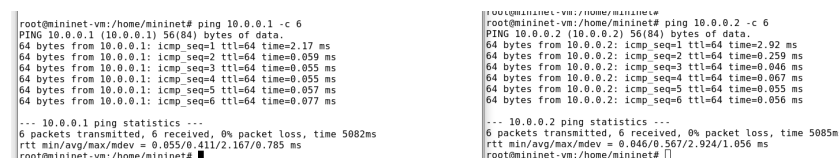
root@mininet-vms:/home/mininet# ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 12:df:97:52:2a:12 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 872 bytes 227064 (227.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 872 bytes 227064 (227.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vms:/home/mininet#
```

Рис. 3.3: Информация на хостах h1 и h2

4. Проверим подключение между хостами h1 и h2 с помощью команды `ping` с параметром `-c 6` (рис. 3.4):



```
root@mininet-vms:/home/mininet# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
 64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.17 ms
 64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.059 ms
 64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.055 ms
 64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.055 ms
 64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.057 ms
 64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.077 ms

--- 10.0.0.1 ping statistics ---
 6 packets transmitted, 6 received, 0% packet loss, time 5082ms
 rtt min/avg/max/mdev = 0.055/0.411/2.167/0.785 ms
root@mininet-vms:/home/mininet#

root@mininet-vms:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
 64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.92 ms
 64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.259 ms
 64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.046 ms
 64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.067 ms
 64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.055 ms
 64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.056 ms

--- 10.0.0.2 ping statistics ---
 6 packets transmitted, 6 received, 0% packet loss, time 5085ms
 rtt min/avg/max/mdev = 0.046/0.567/2.924/1.056 ms
root@mininet-vms:/home/mininet#
```

Рис. 3.4: Проверка подключения между хостами

Для IP-адреса 10.0.0.1:

- Минимальное RTT: 0.055 мс
- Среднее RTT: 0.411 мс
- Максимальное RTT: 2.167 мс
- Стандартное отклонение: 0.785 мс

Для IP-адреса 10.0.0.2:

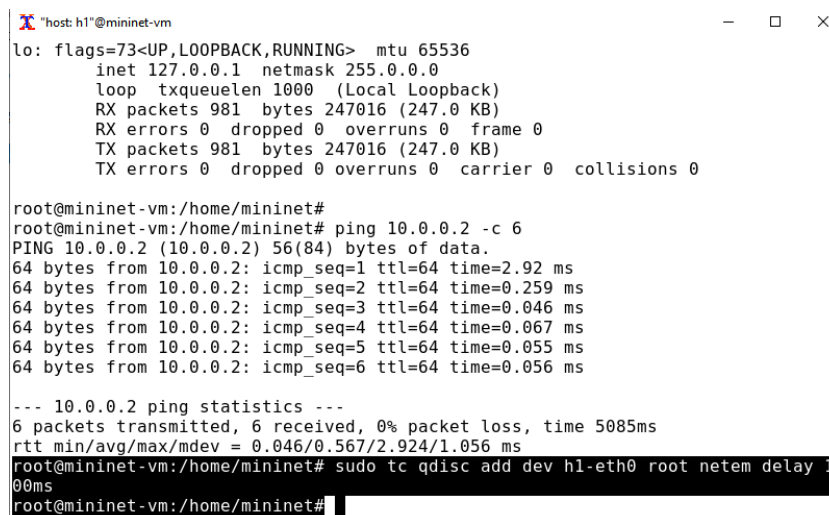
- Минимальное RTT: 0.046 мс
- Среднее RTT: 0.567 мс
- Максимальное RTT: 2.924 мс
- Стандартное отклонение: 1.056 мс

3.2 Интерактивные эксперименты

3.2.1 Добавление/изменение задержки в эмулируемой глобальной сети

Сетевые эмуляторы задают задержки на интерфейсе. Например, задержка, вносимая в интерфейс коммутатора А, который подключён к интерфейсу коммутатора В, может представлять собой задержку распространения WAN, соединяющей оба коммутатора.

1. На хосте h1 добавим задержку в 100 мс к выходному интерфейсу (рис. 3.5):



```

X "host: h1" @mininet-vm
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 981 bytes 247016 (247.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 981 bytes 247016 (247.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet#
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.92 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.259 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.046 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.056 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5085ms
rtt min/avg/max/mdev = 0.046/0.567/2.924/1.056 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms
root@mininet-vm:/home/mininet#

```

Рис. 3.5: Добавление задержки в 100 мс

Здесь:

- `sudo`: выполнить команду с более высокими привилегиями;
- `tc`: вызвать управление трафиком Linux;
- `qdisc`: изменить дисциплину очередей сетевого планировщика;
- `add`: создать новое правило;
- `dev h1-eth0`: указать интерфейс, на котором будет применяться правило;
- `netem`: использовать эмулятор сети;
- `delay 100ms`: задержка ввода 100 мс.

2. Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс, используя команду `ping` с параметром `-c 6` с хоста h1 (рис. 3.6):

```

root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 100.557/101.095/102.207/0.530 ms
root@mininet-vm:/home/mininet#

```

Рис. 3.6: Проверка задержки в 100 мс

- Минимальное RTT: 100.557 мс
- Среднее RTT: 101.095 мс
- Максимальное RTT: 102.207 мс
- Стандартное отклонение: 0.530 мс

3. Для эмуляции глобальной сети с двунаправленной задержкой необходимо к соответствующему интерфейсу на хосте h2 также добавить задержку в 100 миллисекунд (рис. 3.7):

```
host: h2" @mininet-vm
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 872 bytes 227064 (227.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 872 bytes 227064 (227.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.17 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.059 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.055 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.055 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.057 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.077 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5082ms
rtt min/avg/max/mdev = 0.055/0.411/2.167/0.785 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem delay 100ms
root@mininet-vm:/home/mininet#
```

Рис. 3.7: Добавление задержки в 100 мс на втором хосте

3. Проверим, что соединение между хостом h1 и хостом h2 имеет RTT в 200 мс (100 мс от хоста h1 к хосту h2 и 100 мс от хоста h2 к хосту h1), повторив команду ping с параметром -c 6 на терминале хоста h1 (рис. 3.8):

```
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=202 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=203 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=200 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=201 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=201 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=202 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010ms
rtt min/avg/max/mdev = 200.498/201.541/202.786/0.803 ms
root@mininet-vm:/home/mininet#
```

Рис. 3.8: Проверка, что соединение имеет RTT в 200мс

- Минимальное RTT: 100.557 мс
- Среднее RTT: 101.095 мс
- Максимальное RTT: 102.207 мс
- Стандартное отклонение: 0.530 мс

3.2.2 Изменение задержки в эмулируемой глобальной сети

1. Изменим задержку со 100 мс до 50 мс для отправителя h1 и для получателя h2 (рис. 3.9):

```
root@mininet-vm:/home/mininet# sudo tc qdisc change dev h2-eth0 root netem delay 50ms
root@mininet-vm:/home/mininet#
```

Рис. 3.9: Изменение задержки до 50 мс

2. Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс, используя команду ping с параметром -c 6 с терминала хоста h1 (рис. 3.10):

```
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010ms
rtt min/avg/max/mdev = 101.030/101.298/101.900/0.297 ms
root@mininet-vm:/home/mininet#
```

Рис. 3.10: Проверка, что задержка 100 мс

- Минимальное RTT: 101.030 мс
- Среднее RTT: 101.298 мс
- Максимальное RTT: 101.900 мс
- Стандартное отклонение: 0.297 мс

3.2.3 Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети

1. Восстановим конфигурацию по умолчанию, удалив все правила, применённые к сетевому планировщику соответствующего интерфейса (рис. 3.11):

```

root@mininet-vm:/home/mininet# sudo tc qdisc del dev h2-eth0 root netem
root@mininet-vm:/home/mininet#
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet#

```

Рис. 3.11: Восстановление конфигураций

2. Проверим, что соединение между хостом h1 и хостом h2 не имеет явно установленной задержки, используя команду `ping` с параметром `-c 6` с терминала хоста h1 (рис. 3.12):

```

root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.73 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.484 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.198 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.057 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.056 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5097ms
rtt min/avg/max/mdev = 0.056/0.598/2.725/0.962 ms
root@mininet-vm:/home/mininet#

```

Рис. 3.12: Проверка, что соединение не имеет явной задержки

- Минимальное RTT: 0.056 мс
- Среднее RTT: 0.598 мс
- Максимальное RTT: 2.725 мс
- Стандартное отклонение: 0.962 мс

3.2.4 Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети

В сетях нет постоянной задержки. Она может варьироваться в зависимости от других потоков трафика, конкурирующих за тот же путь. Джиттер (jitter) — это изменение времени задержки.

1. При необходимости восстановим конфигурацию интерфейсов по умолчанию на узлах h1 и h2:

2. Добавим на узле h1 задержку в 100 мс со случайным отклонением 10 мс :
`sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms.`
3. Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс со случайным отклонением ± 10 мс, используя в терминале хоста h1 команду `ping` с параметром `-c 6` (рис. 3.13):

```
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=104 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=107 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=109 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt_min/avg/max/mdev = 101.753/105.843/110.242/3.194 ms
```

Рис. 3.13: Проверка, что имеется задержка 100 мс со случайным отклонением ± 10 мс

- Минимальное RTT: 101.753 мс
 - Среднее RTT: 105.843 мс
 - Максимальное RTT: 110.242 мс
 - Стандартное отклонение: 3.194 мс
4. Восстановим конфигурацию интерфейса по умолчанию на узле h1 (рис. 3.14):

```
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet#
```

Рис. 3.14: Восстановление конфигурации на h1

3.2.5 Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети

1. При необходимости восстановим конфигурацию интерфейсов по умолчанию на узлах h1 и h2.
2. Добавим на интерфейсе хоста h1 задержку в 100 мс с вариацией ± 10 мс и значением корреляции в 25%. Убедимся, что все пакеты, покидающие устройство h1 на интерфейсе h1-eth0, будут иметь время задержки 100 мс со случайным отклонением ± 10 мс, при этом время передачи следующего пакета зависит от предыдущего значения на 25% (рис. 3.15):

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms 25%
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 20
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=92.4 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=91.2 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=91.5 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=91.4 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=98.3 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=92.1 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=90.7 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=99.7 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=104 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=99.6 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=97.2 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=107 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19042ms
rtt min/avg/max/mdev = 90.741/99.104/107.297/5.490 ms
root@mininet-vm:/home/mininet#
```

Рис. 3.15: Проверка, что есть задержка в 100 мс с вариацией ± 10 мс и значением корреляции в 25%

- Минимальное RTT: 90.741 мс
- Среднее RTT: 99.104 мс
- Максимальное RTT: 107.297 мс

- Стандартное отклонение: 5.490 мс

3. Восстановим конфигурацию интерфейса по умолчанию на узле h1.

3.2.6 Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети

NETEM позволяет пользователю указать распределение, которое описывает, как задержки изменяются в сети. В реальных сетях передачи данных задержки неравномерны, поэтому при моделировании может быть удобно использовать некоторое случайное распределение, например, нормальное, парето или парето- нормальное.

1. При необходимости восстановим конфигурацию интерфейсов по умолчанию на узлах h1 и h2. Убедимся, что все пакеты, покидающие хост h1 на интерфейсе h1-eth0, будут иметь время задержки, которое распределено в диапазоне 100 мс $\pm$ 20 мс. Используем для этого команду ping на терминале хоста h1 с параметром -c 10 (рис. 3.16):

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 20ms distribution normal
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 10
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=119 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=89.3 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=76.0 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=126 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=122 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=126 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=68.8 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=74.5 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=119 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=118 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9016ms
rtt min/avg/max/mdev = 68.759/103.876/126.065/22.479 ms
root@mininet-vm:/home/mininet#
```

Рис. 3.16: Проверка, что есть задержка, которая распределена в диапазоне 100 мс $\pm$ 20 мс

2. Восстановим конфигурацию интерфейса по умолчанию на узле h1.
3. Завершим работу mininet в интерактивном режиме, введя в интерфейсе mininet (рис. 3.17):

```
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 8 terms
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 1312.958 seconds
mininet@mininet-vm:~$
```

Рис. 3.17: Завершение работы mininet

3.3 Воспроизведение экспериментов

3.3.1 Предварительная подготовка

1. Обновим репозитории программного обеспечения на виртуальной машинеВ (рис. 3.18):

```
mininet@mininet-vm:~$ sudo apt-get update
Hit:1 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
Reading package lists... Done
mininet@mininet-vm:~$
```

Рис. 3.18: Обновление ПО

2. Установим пакет `gexif` — понадобится для просмотра файлов `png` (рис. 3.19):

```

Setting up aptdaemon (1.1.1+bzr982-0ubuntu32.3) ...
Setting up python3-aptdaemon (1.1.1+bzr982-0ubuntu32.3) ...
Setting up python3-aptdaemon.gtk3widgets (1.1.1+bzr982-0ubuntu32.3) ...
Setting up language-selector-gnome (0.204.2) ...
Setting up gnome-control-center (1:3.36.5-0ubuntu4.1) ...
Processing triggers for mime-support (3.64ubuntu1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for libgtk-3-0:amd64 (3.24.20-0ubuntu1) ...
Processing triggers for libc-bin (2.31-0ubuntu9) ...
Processing triggers for systemd (245.4-4ubuntu3.4) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for shared-mime-info (1.15-1) ...
Processing triggers for udev (245.4-4ubuntu3.4) ...
Processing triggers for fontconfig (2.13.1-2ubuntu3) ...
Processing triggers for sgml-base (1.29.1) ...
Setting up sgml-data (2.0.11) ...
Processing triggers for sgml-base (1.29.1) ...
Setting up docbook-xml (4.5-9) ...
Processing triggers for dbus (1.12.16-2ubuntu2.3) ...
Processing triggers for rygel (0.38.3-1ubuntu1) ...
Processing triggers for sgml-base (1.29.1) ...
mininet@mininet-vm:~$

```

Рис. 3.19: Установка пакета geeqie

3. Для каждого воспроизводимого эксперимента exrname создайте свой каталог, в котором будут размещаться файлы эксперимента (рис. 3.20):

```

mininet@mininet-vm:~$ mkdir -p ~/work/lab_netem_i/exrname
mininet@mininet-vm:~$

```

Рис. 3.20: Создание каталога exrname

Здесь exrname может принимать значения simple-delay, change-delay, jitter-delay, correlation-delay и т.п.

Для каждого случая создадим скрипт для проведения эксперимента lab_netem_i.py и скрипт для визуализации результатов ping_plot.

3.3.2 Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети

С помощью API Mininet воспроизведите эксперимент по добавлению задержки для интерфейса хоста, подключающегося к эмулируемой глобальной сети.

1. В виртуальной среде mininet в своём рабочем каталоге с проектами создадим каталог simple-delay и перейдем в него (рис. 3.21):

```
mininet@mininet-vm:~$ mkdir -p ~/work/lab_netem_i/simple-delay
mininet@mininet-vm:~$ cd ~/work/lab_netem_i/simple-delay
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ nano lab_netem_i.py
```

Рис. 3.21: Создание каталога simple-delay

2. Создаем скрипт для эксперимента lab_netem_i.py (рис. 3.22):

```
GNU nano 4.8 lab_netem_i.py Modified
#!/usr/bin/env python

"""
Simple experiment.
Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI

from mininet.log import setLogLevel, info
import time

def emptyNet():
    "Create an empty network and add nodes to it."
    net = Mininet(controller=Controller, waitConnected=True)
    info('*** Adding controller\n')
    net.addController('c0')
    info('*** Adding hosts\n')
    h1 = net.addHost('h1', ip='10.0.0.1')
    h2 = net.addHost('h2', ip='10.0.0.2')
    info('*** Adding switch\n')
    s1 = net.addSwitch('s1')
    info('*** Creating links\n')
    net.addLink(h1, s1)
    net.addLink(h2, s1)
    info('*** Starting network\n')
    net.start()
    info('*** Set delay\n')
    h1.cmdPrint('tc qdisc add dev h1-eth0 root netem delay 100ms')
    h2.cmdPrint('tc qdisc add dev h2-eth0 root netem delay 100ms')
    time.sleep(10) # Wait 10 seconds
    info('*** Ping\n')
    h1.cmdPrint('ping -c 100', h2.IP(), '| grep "time=" | awk \'{print $5, $7}\''
    info('*** Stopping network')
    net.stop()

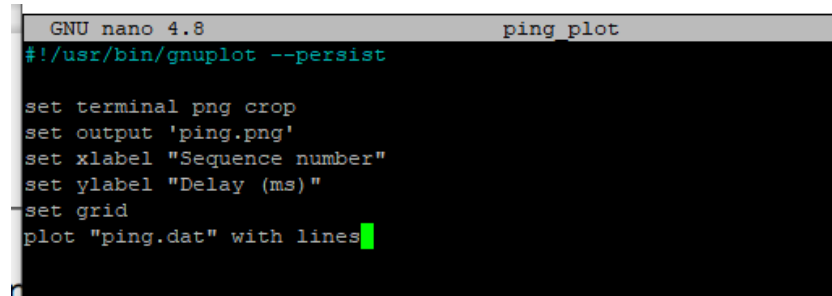
if __name__ == '__main__':
    setLogLevel('info')
    emptyNet()
```

Рис. 3.22: Создание скрипта lab_netem_i.py

Скрипт создает простую сетевую топологию в Mininet с двумя хостами и одним коммутатором, настраивает задержку и выполняет ping-тест для сбора данных. Задержка задается в следующих строках:

```
h1.cmdPrint('tc qdisc add dev h1-eth0 root netem delay 100ms')
h2.cmdPrint('tc qdisc add dev h2-eth0 root netem delay 100ms')
```

3. Создаём скрипт для визуализации ping_plot результатов эксперимента (рис. 3.23):

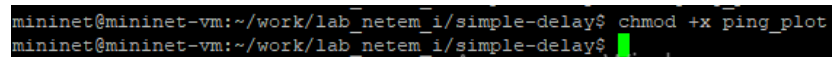


```
GNU nano 4.8                                ping_plot
#!/usr/bin/gnuplot --persist

set terminal png crop
set output 'ping.png'
set xlabel "Sequence number"
set ylabel "Delay (ms)"
set grid
plot "ping.dat" with lines
```

Рис. 3.23: Создание скрипта lab_netem_i.py

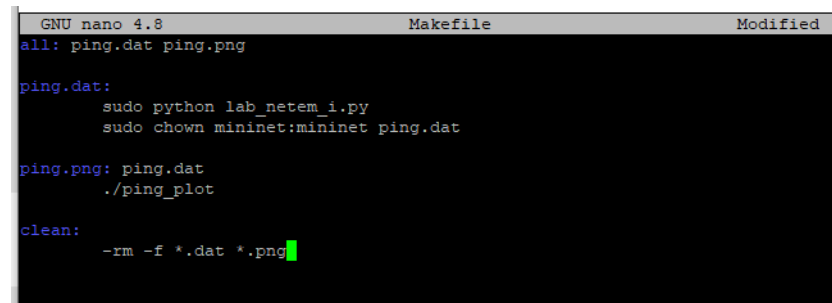
4. Зададим права доступа к файлу скрипта (рис. 3.24):



```
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ chmod +x ping_plot
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$
```

Рис. 3.24: Задание прав доступа

5. Создадим Makefile для управления процессом проведения эксперимента (рис. 3.25):



```
GNU nano 4.8                                Makefile                                Modified
all: ping.dat ping.png

ping.dat:
    sudo python lab_netem_i.py
    sudo chown mininet:mininet ping.dat

ping.png: ping.dat
    ./ping_plot

clean:
    -rm -f *.dat *.png
```

Рис. 3.25: Создание Makefile

6. Запустим с помощью make и получим следующий график (рис. 3.26):

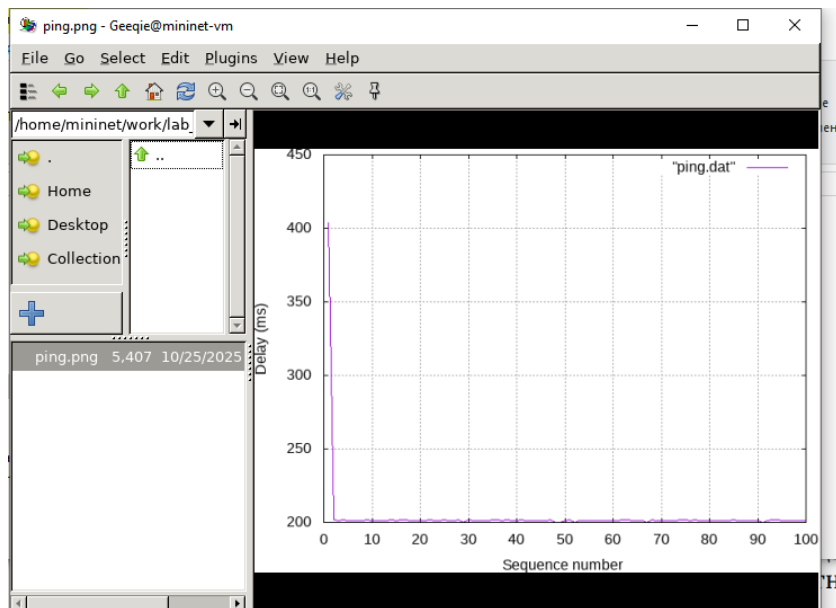


Рис. 3.26: График ping.png

```
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ make
make: Nothing to be done for 'all'.
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ make clean
rm -f *.dat *.png
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms',)
*** Ping
*** h1 : ('ping -c 100 10.0.0.2 | grep "time=" | awk \'{print $7, $8}\'} | sed \'
s/time=//g\' | sed \'/s/icmp_seq=//g\' > ping.dat',)
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
./ping_plot
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ geeqie ping.png
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$
```

Рис. 3.27: Запуск

7. Из файла ping.dat удалим первую строку и заново построим график (рис.

3.28):

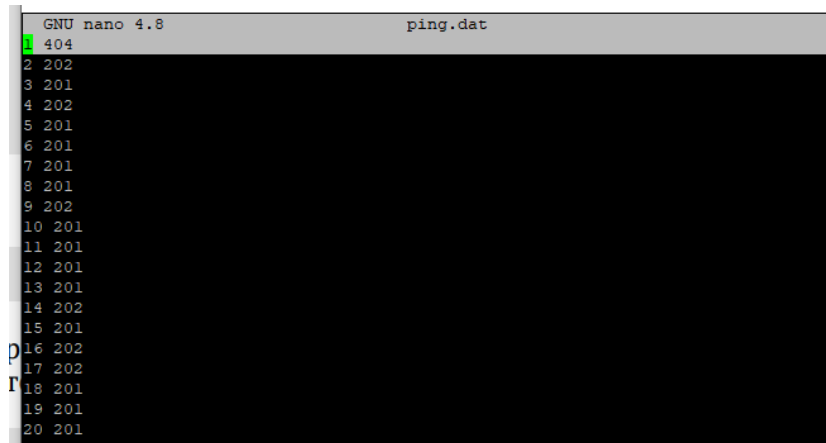


Рис. 3.28: Удаление первой строчки

8. Выведем снова график с учетом изменений (рис. 3.29):

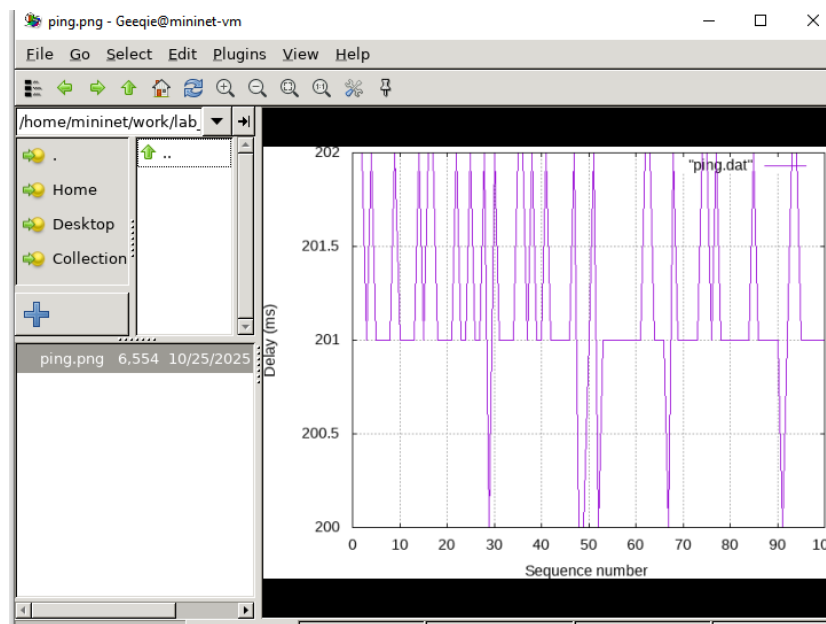


Рис. 3.29: График ping.png с учетом изменений

9. Разработаем скрипт для вычисления на основе данных файла ping.dat минимального, среднего, максимального и стандартного отклонения времени приёма-передачи (рис. 3.30):

```

with open('ping.dat', 'r') as f:
    s = []
    for line in f.readlines():
        if '\n' in line:
            line.replace('\n', "")
            s.append([int(j) for j in (line.split(" "))])
    s = [j[1] for j in s]
    std = (sum([(i-(sum(s)/len(s)))**2 for i in s])/(len(s)-1))**0.5
    print(f"min: {min(s)} \nmax: {max(s)} \navg: {sum(s)/len(s)} \nstd: {std}")

```

Рис. 3.30: Скрипт для вычисления на основе ping.dat

```

mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ sudo python3 script.py
min: 200
max: 202
avg: 201.1919191919192
std: 0.528315916558584

```

Рис. 3.31: Запуск скрипта для вычисления

3.3.3 Задание для самостоятельной работы

Самостоятельно реализуем воспроизводимые эксперименты по изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети. Построим графики. Вычислим минимальное, среднее, максимальное и стандартное отклонение времени приёма-передачи для каждого случая (рис. 3.32):

```

GNU nano 4.8                                lab_netem_i.py                                Modified
#!/usr/bin/env python

"""
Simple experiment.
Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():
    "Create an empty network and add nodes to it."

    net = Mininet(controller=Controller, waitConnected=True)

    info('*** Adding controller\n')
    net.addController('c0')

    info('*** Adding hosts\n')
    h1 = net.addHost('h1', ip='10.0.0.1')
    h2 = net.addHost('h2', ip='10.0.0.2')

    info('*** Adding switch\n')
    s1 = net.addSwitch('s1')

    info('*** Creating links\n')
    net.addLink(h1, s1, delay='50ms', jitter='10ms')
    net.addLink(h2, s1, delay='50ms', jitter='10ms')

```

Рис. 3.32: Внесение изменений в lab_netem_i.py

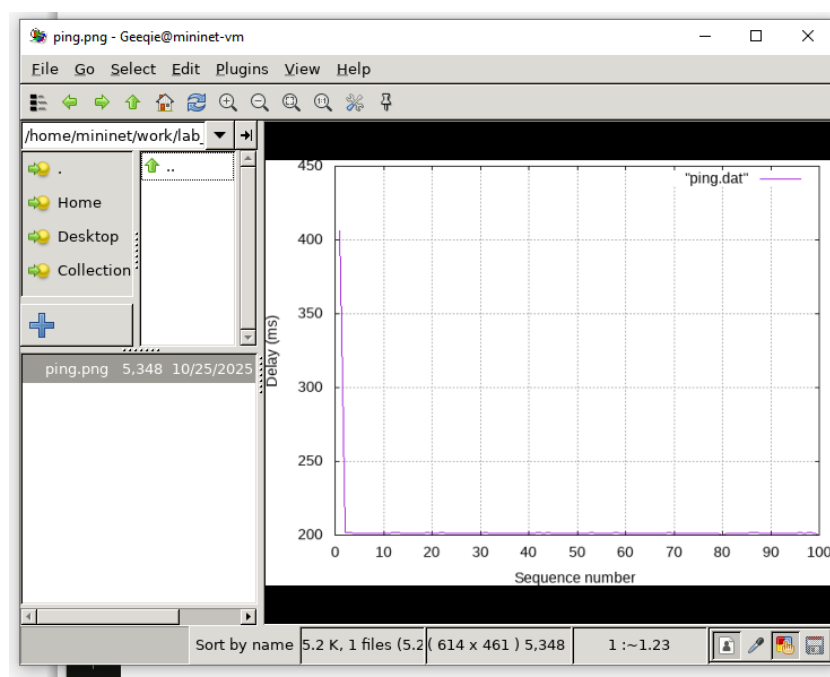


Рис. 3.33: Запуск обновленного скрипта с выводом графика

```
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ sudo python3 script.py
min: 200
max: 406
avg: 203.19
std: 20.489807764403796
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$
```

Рис. 3.34: Запуск script.py на основе обновленного ping.dat

4 Выводы

В результате выполнения данной лабораторной работы я познакомилась с NETEM — инструментом для тестирования производительности приложений в виртуальной сети, а также получила навыков проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.